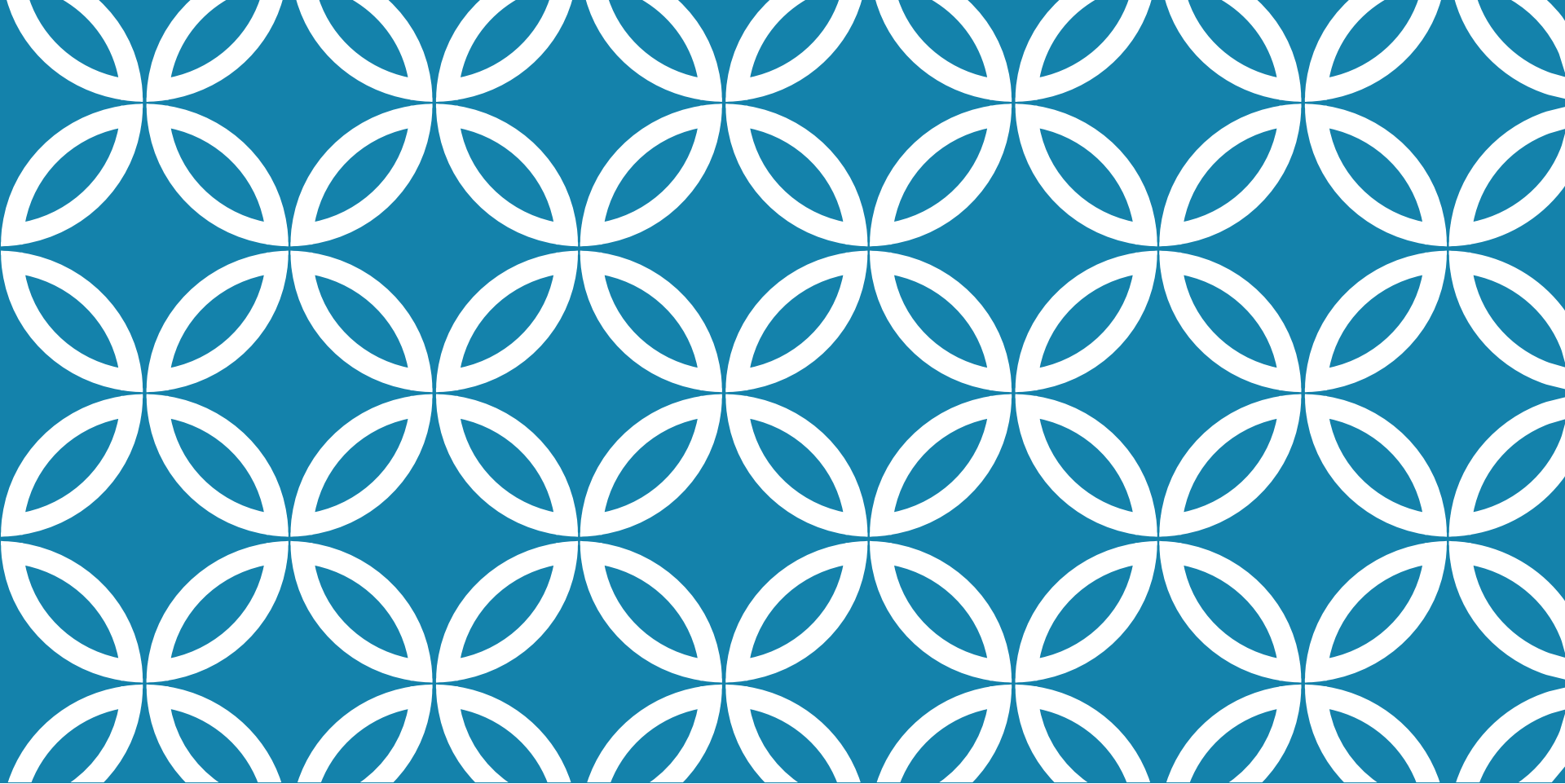




LECTURE 5.1: FILES AND COMPONENTS OF XV6-RISCV

CSE321: Operating
Systems Laboratory

OBJECTIVES

- Understand the overall structure of the xv6-riscv operating system.
- Identify the major directories, files, and components of xv6-riscv.
- Explain, at a high level, how xv6 boots, creates processes, handles system calls, and manages memory.
- Relate user programs and system calls to the corresponding kernel files and layers (processes, traps, file system, devices).

BIG PICTURE: USER SPACE VS KERNEL SPACE

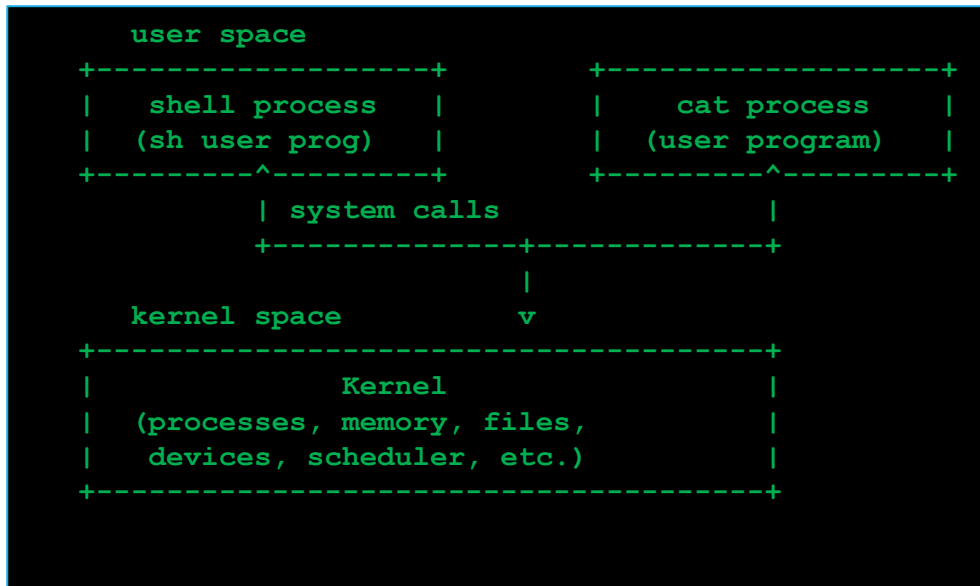


Diagram Reference: [xv6 RISC-V book, Chapter 1 \(Overview and kernel/user split\)](#)

What is xv6?

xv6 is a small teaching operating system used in university operating systems courses. It reimplements an early version of Unix (Version 6) in modern C and RISC-V assembly. The xv6-riscv code is intentionally small and readable, which makes it suitable for students who are seeing OS internals for the first time.

Core ideas:

- **Kernel** runs in supervisor mode and directly controls the CPU, memory, and devices.
- The **user programs** run in user mode with restricted privileges.
- **User programs** invoke **system calls** to request services from the kernel.

TOP-LEVEL LAYOUT OF XV6-RISCV

```
xv6-riscv/
├── Makefile           (build rules)
├── mkfs               (builds fs.img)
├── kernel/            (OS kernel, runs in supervisor mode)
│   ├── entry.S
│   ├── start.c
│   ├── main.c
│   ├── proc.c, exec.c, swtch.S
│   ├── trap.c, syscall.c, ...
│   ├── vm.c, kalloc.c
│   ├── fs.c, file.c, log.c, bio.c, ...
│   └── drivers: uart.c, console.c, virtio_disk.c, plic.c, ...
└── user/              (user programs + tiny libc)
    ├── init.c, sh.c
    ├── ls.c, cat.c, echo.c, ...
    ├── user.h, ulib.c, usys.S, printf.c
    └── tests: usertests.c, etc.
```

Diagram Reference: [xv6 RISC-V book, Sections 2.1–2.4 \(Code organisation and file layout\)](#)

Important elements in the top-level directory:

- **kernel/** Contains the operating system kernel source files (C and assembly).
- **user/** Contains user programs (such as sh, ls, cat) and a tiny user-space library.
- **mkfs:** A tool that builds the initial xv6 file system image (fs.img) from a set of files.
- **Makefile:** It describes how to build xv6 and its user programs, as well as how to run xv6 under QEMU.
- **Other files** include Helper scripts, configuration files, and documentation (e.g., README).

BOOT PATH/SEQUENCE: FROM HARDWARE TO SHELL

```
Hardware / QEMU
|
| (boot ROM + boot loader, outside xv6)
v
[1] entry.S          (kernel/entry.S)
    - sets up stack
    - jumps to start()
    |
    v
[2] start.c          (kernel/start.c)
    - runs in machine mode
    - configures timer, interrupts
    - switches to supervisor mode
    - calls main()
    |
    v
[3] main.c           (kernel/main.c)
    - initialises memory, process table, file system,
    devices
    - calls userinit() -> start user program 'init'
    - enters scheduler loop
    |
    v
[4] init (user/init.c)
    - creates console, starts shell 'sh'
    |
    v
[5] sh (user/sh.c)
    - reads commands, runs other programs
```

Key function: A small chain of files enables xv6 to transition from raw hardware to a running shell.

- **kernel/entry.S:** First kernel instructions, written in RISC-V assembly; sets up a stack and jumps to start().
- **kernel/start.c:** Runs in machine mode, configures timer and interrupt delegation, switches to supervisor mode, then calls main().
- **kernel/main.c:** Initializes memory, processes, file system, devices, starts first user process (init), and enters the scheduler.
- **user/init.c:** First user program, sets up console and starts sh.
- **user/sh.c:** Shell that reads commands, uses fork and exec to run programs.

Diagram Reference: [xv6 RISC-V book, Section 2.6 \(Starting xv6 and the first process\)](#)

PROCESSES AND SYSTEM CALLS

System call path (example: write)

```
User program (e.g., sh or ls)
|
v
write(fd, buf, n)           // user C call
|
v
usys.S                     // ECALL with syscall number
|
v
trampoline.S, trap.c       // trap into kernel, identify syscall
|
v
syscall.c -> sys_write()    // perform write via file system
|
v
return to user             // write() returns value
```

Diagram Reference: [xv6 RISC-V book, Chapters 2–3 \(Processes and address spaces\)](#)

Key files:

- **kernel/proc.c:** Process table, process states, scheduler, logic for fork, exit, wait.
- **kernel/exec.c:** Implements exec(); loads a new program (ELF file) into a process.
- **kernel/switch.S:** Assembly routine for context switching between processes.
- **kernel/sysproc.c:** Kernel implementations of process-related system calls.

MEMORY AND FILE SYSTEM (HIGH-LEVEL VIEW)

Simple file system stack for read()

```
read(fd, buf, n)
  |
  v
sys_read() (sysfile.c)
  |
  v
fileread() (file.c)      -> inode
  |
  v
readi() (fs.c)           -> disk block numbers
  |
  v
bread() (bio.c)          -> virtio_disk.c -> disk
```

Diagram Reference: [xv6 RISC-V book, Chapters 8–10 \(File system structure and logging\)](#)

Memory:

- **kernel/kalloc.c:** Physical page allocator (kalloc, kfree).
- **kernel/vm.c:** Virtual memory and page tables (maps code, data, stack, trampoline).

File System:

- **kernel/fs.c, kernel/bio.c, kernel/log.c:** Disk layout, buffer cache, simple log.
- **kernel/file.c, kernel/sysfile.c, kernel/pipe.c:** File descriptors, file system system calls, pipes.

USER-SPACE FILES AND PROGRAMS (USER/DIRECTORY)

User Program Calling a System Call

```
user program (e.g., ls.c)
  |
  v
user.h + ulib.c  (write(), read(), open(), ...)
  |
  v
usys.S stubs    (assembly ECALL wrapper)
  |
  v
kernel (syscall.c + sys*.c)
```

Diagram Reference: xv6 RISC-V book, Chapters 1–2 and 4 (User programs and system call interface)

Supporting Code:

- **user.h**: Contains function prototypes and type definitions for user programs, including system call declarations.
- **ulib.c**: Implements basic C library functions for user programs.
- **printf.c**: User-space printf implementation.
- **usys.S**: Assembly stubs that place the system call number in a register, execute ecall, and return the result.

Important programs:

- **init.c**: The first user program started by the kernel. It sets up the console and then starts the shell (sh).
- **sh.c**: The shell program. It reads commands from the user, uses fork() and exec() to run other programs, and waits for them to finish.
- **Various utilities** (ls, cat, echo, mkdir, rm, kill, sleep, etc.) and **test programs** (usertests).

QUICK FILE MAPPING: WHERE TO LOOK IN XV6-RISCV

- **Overall structure and files:** top-level tree and kernel/ vs user/
- **Boot:** kernel/entry.S, kernel/start.c, kernel/main.c
- **Processes and scheduler:** kernel/proc.c, kernel/exec.c, kernel/switch.S
- **Traps and system calls:** kernel/trampoline.S, kernel/trap.c, kernel/syscall.c, kernel/sysproc.c, kernel/sysfile.c
- **Memory and address spaces:** kernel/vm.c, kernel/kalloc.c
- **File system and storage:** kernel/fs.c, kernel/bio.c, kernel/log.c, kernel/file.c, kernel/pipe.c
- **Devices and interrupts:** kernel/uart.c, kernel/console.c, kernel/plic.c, kernel/virtio_disk.c
- **User level:** user/init.c, user/sh.c, user/*.c utilities, user/user.h, user/ulib.c, user/usys.S

REFERENCES AND FURTHER READING

- Overview of xv6-riscv design, file layout, and code examples: [xv6: a simple, Unix-like teaching operating system \(RISC-V edition\)](#)
- Official xv6-riscv source code repository (all files mentioned here): [mit-pdos/xv6-riscv on GitHub](#)
- Course materials, labs, and assignments using xv6-riscv: [MIT 6.S081 / 6.1810 Operating Systems Engineering](#)
- Background on RISC-V privilege levels, page tables, and traps: [RISC-V Instruction Set Architecture Specifications](#)
- Emulator used to run xv6-riscv: [QEMU official website](#)